Complete In-Depth Guide: Arrays & Strings in Java

TABLE OF CONTENTS

1. Why Arrays Exist (The Problem)
2. What is an Array (The Solution)
3. Array Declaration & Definition
4. Memory Allocation (How it actually works)
5. Indexes in Arrays
6. Inserting Values into Arrays
7. Retrieving Values from Arrays
8. Using Loops with Arrays
9. The `.length` Property
10. IndexOutOfBoundsException
11. Multi-Dimensional Arrays
12. Conceptual vs Logical Representation
13. Jagged Arrays (Different Length Rows)
14. 3D & Higher Dimensional Arrays
15. Shorthand Array Declaration
16. Introduction to Strings

---

1. WHY ARRAYS EXIST — THE PROBLEM

Imagine you're building a program to store roll numbers of students. The "naive" approach would be:

```
int rollNumber0 = 1001;
int rollNumber1 = 1002;
int rollNumber2 = 1003;
int rollNumber3 = 1004;
// ... and so on
```

What's wrong with this?

- If you have **100 students**, you write 100 separate variable declarations
- If you have **10,000 students**, you write 10,000 lines — completely impractical

- You cannot loop over individual variables — there's no way to say "go through all roll numbers automatically"
- Every new student requires **manually editing source code**
- Managing, updating, or searching these becomes a nightmare

This is the core problem. You need a way to store a **collection** of values of the same type, under a **single name**, and be able to access them systematically.

---

2. WHAT IS AN ARRAY — THE SOLUTION

An **array** is a data structure that stores a **fixed-size sequential collection of elements of the same data type** in **contiguous (adjacent) memory locations**.

Let's break every word of that definition down:

"Fixed-size"

Once you create an array of size 5, it will **always** be size 5. You cannot shrink it to 3 or grow it to 10. The size is locked at creation time. (For dynamic sizing, Java has ArrayList, but that's separate.)

"Sequential collection"

The elements are stored one after another, not scattered randomly. Element 0, then element 1, then element 2 — in order.

"Same data type"

All elements must be of the same type. An `int` array holds only integers. A `double` array holds only doubles. You cannot mix `int` and `String` in a single array.

"Contiguous memory locations"

This is the most important part for understanding how arrays work internally. Instead of this (separate, scattered variables):

```
Memory location 100: rollNumber0 = 1001
Memory location 500: rollNumber1 = 1002   ← completely different location!
Memory location 230: rollNumber2 = 1003   ← again scattered
```

An array forces memory to be allocated like this:

Memory location 100: 1001 ← index 0
Memory location 104: 1002 ← index 1 (4 bytes later, since int = 4 bytes = 32 bits)
Memory location 108: 1003 ← index 2 (4 bytes after that)

Why does contiguity matter?

Because the CPU can calculate the exact memory address of any element using a simple formula:

Address of element[i] = Base Address + (i × size of one element)

For example: Base address = 100, element size = 4 bytes (int)

- element[0] → 100 + (0 × 4) = **100**
- element[1] → 100 + (1 × 4) = **104**
- element[5] → 100 + (5 × 4) = **120**

This is why accessing any element of an array is **O(1)** — constant time. The computer doesn't "search" for it; it **calculates** where it is directly.

---

3. ARRAY DECLARATION & DEFINITION

In Java, working with a variable has two steps: Declaration and Definition. Same applies to arrays.

Declaration (just announcing it exists)

```
int[] rollNumbers;
```

Or equivalently (older C-style, also valid in Java):

```
int rollNumbers[];
```

What happens here?

- You tell the compiler: "Hey, I'm going to use a variable called `rollNumbers` that will hold an array of integers."
- **No memory is allocated yet.** The variable just exists as a reference (pointer) that currently points to nothing (`null`).

- Think of it like saying "I'm going to buy a house" — you haven't bought it yet, you've just made a statement of intent.

Why does Java allow both `int[] name` and `int name[]`?

- `int[] name` is the **Java-preferred** style. The `[]` is part of the type, making it clear "the type is int-array."
- `int name[]` is the **C/C++ legacy** style. Java supports it for backwards compatibility with programmers coming from C.
- They compile to identical bytecode. No functional difference.

Definition (actually creating it in memory)

rollNumbers = new int[3];

Or declaration + definition together (most common):

int[] rollNumbers = new int[3];

Breaking down `new int[3]` piece by piece:

Part	What it means
`new`	Special keyword — triggers memory allocation in the **Heap**
`int`	The data type of each element in this array
`[3]`	The number of elements (size/capacity) of this array

What exactly is the `new` keyword doing?

`new` instructs the JVM (Java Virtual Machine) to:

1. Go to the **Heap memory** (a large pool of memory managed by JVM)
2. Find a contiguous block large enough for 3 integers
3. Since each `int` is 32 bits (4 bytes), allocate 3 × 4 = **12 bytes** of contiguous memory
4. Initialize all 3 slots to **default value** (for `int`, default is `0`)
5. Return the **memory address** of the first slot
6. Store that address in the variable `rollNumbers`

Stack vs Heap — a critical concept:

```
STACK MEMORY:                    HEAP MEMORY:

┌─────────────────┐              ┌──────────────────────────┐
│ rollNumbers     │──────────────│ [0] = 0  (address: 1000) │
│ (holds address  │              │ [1] = 0  (address: 1004) │
│  1000)          │              │ [2] = 0  (address: 1008) │
└─────────────────┘              └──────────────────────────┘
```

- The **variable `rollNumbers`** itself lives on the Stack and holds a memory address (a reference/pointer to the heap)
- The **actual array data** lives on the Heap
- This is why arrays are called **reference types** (non-primitive) in Java

This is a fundamental distinction between primitive types and arrays:

int x = 5; // x lives on Stack, holds the VALUE 5 directly
int[] arr = new int[3]; // arr lives on Stack, holds an ADDRESS pointing to Heap

---

4. MEMORY ALLOCATION IN DEPTH

When you write `int[] arr = new int[5]`, here's the complete sequence of events:

Step 1: JVM sees `new int[5]`

Step 2: JVM calculates memory needed:

5 elements × 32 bits per int = 160 bits = 20 bytes

Step 3: JVM finds 20 bytes of **contiguous free space** in Heap memory

Step 4: JVM fills all 5 slots with the default value for `int`, which is `0`:

Heap:
Address 2000: 0 ← arr[0]
Address 2004: 0 ← arr[1]
Address 2008: 0 ← arr[2]
Address 2012: 0 ← arr[3]
Address 2016: 0 ← arr[4]

Step 5: JVM stores the base address `2000` in the Stack variable `arr`

Default values by type:

Type	Default Value
`int`, `byte`, `short`, `long`	`0`
`float`, `double`	`0.0`
`boolean`	`false`
`char`	`'\u0000'` (null character)
Objects, arrays, Strings	`null`

---

5. INDEXES IN ARRAYS — THE ADDRESSING SYSTEM

Once an array exists in memory, you need a way to refer to specific elements. This is done through **indexes** (also called indices).

Zero-Based Indexing

Java uses **zero-based indexing**, meaning:

- The **first** element is at index **0** (not 1)
- The **second** element is at index **1**
- The **last** element of an array of size `n` is at index **n-1**

```
int[] arr = new int[5];
//          Index:  0   1   2   3   4
//          Value: [0] [0] [0] [0] [0]
```

Why does indexing start at 0?

This goes back to the memory address calculation formula:

Address of element[i] = Base Address + (i × element_size)

If the first element is at index 0:

Address of element[0] = Base + (0 × 4) = Base + 0 = Base ✓ (the very first address)

If indexing started at 1:

Address of element[1] = Base + (1 × 4) = Base + 4 ✗ (skips the first slot!)

You'd waste a slot, or need a more complex formula. Zero-indexing makes the formula clean and efficient. It's a convention inherited from C/C++ and adopted by most modern languages.

Valid Index Range

For an array of size `n`:

- Valid indexes: `0` to `n-1`
- **Invalid:** Any index `< 0` or `>= n`

```
int[] arr = new int[3];  // size = 3
arr[0] = 10;  // valid
arr[1] = 20;  // valid
arr[2] = 30;  // valid
arr[3] = 40;  // INVALID! Index 3 doesn't exist → Exception!
arr[-1] = 50; // INVALID! Negative indexes don't exist → Exception!
```

---

6. INSERTING VALUES INTO ARRAYS

Method 1: One by one (manual assignment)

```
int[] rollNumbers = new int[3];

rollNumbers[0] = 1001;  // Put 1001 at index 0
rollNumbers[1] = 1002;  // Put 1002 at index 1
rollNumbers[2] = 1003;  // Put 1003 at index 2
```

Syntax breakdown: `arrayName[index] = value;`

- `arrayName` — the array you created
- `[index]` — which position to write to (0-based)

- `= value` — what value to put there

Memory after these assignments:

Heap:
Address 1000: 1001 ← rollNumbers[0]
Address 1004: 1002 ← rollNumbers[1]
Address 1008: 1003 ← rollNumbers[2]

Method 2: Using a for loop (when values follow a pattern)

```
int[] rollNumbers = new int[3];
int x = 1001;

for (int i = 0; i < 3; i++) {
    rollNumbers[i] = x;
    x++;
}
```

Trace through execution:

Iteration	i	x (before assignment)	rollNumbers[i]	x (after x++)
1st	0	1001	rollNumbers[0] = 1001	1002
2nd	1	1002	rollNumbers[1] = 1002	1003
3rd	2	1003	rollNumbers[2] = 1003	1004
—	3	—	Loop condition `i < 3` is false, loop exits	—

Method 3: Shorthand initialization (declare + define + assign at once)

```
int[] rollNumbers = {1001, 1002, 1003};
```

What happens here:

- No `new` keyword needed — Java infers it
- No size needed — Java counts the values you provided (3 values → size 3)
- Values are assigned immediately at each index in order
- `rollNumbers[0]` = 1001, `rollNumbers[1]` = 1002, `rollNumbers[2]` = 1003

This is called an **array initializer** or **array literal**.

---

7. RETRIEVING VALUES FROM ARRAYS

Reading a single element

```
System.out.println(rollNumbers[0]);  // prints 1001
System.out.println(rollNumbers[1]);  // prints 1002
System.out.println(rollNumbers[2]);  // prints 1003
```

How this works internally:

1. JVM looks up what address `rollNumbers` points to (say, address 1000)
2. To get `rollNumbers[1]`, JVM calculates: `1000 + (1 × 4) = 1004`
3. JVM reads the 4 bytes at address 1004 → gets `1002`
4. Prints `1002`

Reading all elements using a loop

```
for (int i = 0; i < rollNumbers.length; i++) {
    System.out.println(rollNumbers[i]);
}
```

Enhanced for loop (for-each) — alternative:

```
for (int roll : rollNumbers) {
    System.out.println(roll);
}
```

This reads as "for each `int` named `roll` in `rollNumbers`". Simpler, but you don't get the index `i`.

---

8. THE `.length` PROPERTY

Every array in Java has a built-in property called `.length` that stores the size of the array.

```
int[] arr = new int[5];
```

```
System.out.println(arr.length);  // prints 5
```

Why use `.length` instead of hardcoding the size?

Bad practice (hardcoding):

```
for (int i = 0; i < 3; i++) {  // What if you later change to size 10?
    System.out.println(arr[i]);  // You'd have to find and change this 3 too
}
```

Good practice (using .length):

```
for (int i = 0; i < arr.length; i++) {  // Automatically adapts to any size
    System.out.println(arr[i]);
}
```

If you change the array from `new int[3]` to `new int[100]`, the loop automatically handles all 100 elements. You change one place, not two.

Important: `.length` is NOT a method!

```
arr.length    // CORRECT — it's a field/property (no parentheses)
arr.length()  // WRONG — arrays don't have a length() method
              // (Strings have length() method, arrays don't)
```

This trips up many beginners. Remember:

- **Arrays:** `array.length` (property, no parentheses)
- **Strings:** `string.length()` (method, with parentheses)
- **ArrayList:** `list.size()` (method, with parentheses)

What `.length` actually is

When you create `new int[5]`, the JVM stores the number `5` alongside the array object in Heap memory. The `.length` field reads that stored value. It's part of every array object automatically.

---

9. IndexOutOfBoundsException — IN DEPTH

What is an Exception?

An **exception** is Java's way of signaling that something went wrong during program execution. It's not a compilation error (those catch syntax mistakes). An exception is a **runtime error** — the code compiled fine, but something went wrong when it actually ran.

What is IndexOutOfBoundsException?

Specifically, for arrays, Java throws an `ArrayIndexOutOfBoundsException` when you try to access an index that doesn't exist.

```
int[] arr = new int[3];  // Valid indexes: 0, 1, 2

arr[0] = 10;   // Fine
arr[3] = 40;   // BOOM! ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
arr[-1] = 50;  // BOOM! ArrayIndexOutOfBoundsException: Index -1 out of bounds for length 3
```

What does the error message look like?

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
Index 3 out of bounds for length 3
    at Main.main(Main.java:6)
```

Reading this error:

- `java.lang.ArrayIndexOutOfBoundsException` — the type of exception
- `Index 3 out of bounds for length 3` — you tried index 3, but max valid is 2 (length-1)
- `at Main.main(Main.java:6)` — it happened in the `main` method, at line 6

Common causes:

```
int[] arr = new int[5];

// ❌ Off by one — forgetting last valid index is length-1
for (int i = 0; i <= arr.length; i++) {  // Should be i < arr.length
    arr[i] = i;  // When i = 5, arr[5] doesn't exist!
}

// ❌ Using wrong index variable
arr[arr.length] = 10;  // arr.length = 5, so this tries arr[5] — doesn't exist!

// ✅ Correct
```

```
for (int i = 0; i < arr.length; i++) {  // i goes 0,1,2,3,4 — all valid
    arr[i] = i;
}
```

Why doesn't Java prevent this at compile time?

Because the index might be a variable or expression computed at runtime. The compiler can't always know what value `i` will have when the program runs.

---

10. MULTI-DIMENSIONAL ARRAYS — COMPLETE DEEP DIVE

The Problem That Motivates 2D Arrays

Let's say you have 3 students, each with marks in 3 subjects:

Student	Hindi	English	Computer Science
S1	50	30	90
S2	60	40	80
S3	70	50	70

With 1D arrays, you'd need **one separate array per student**:

```
int[] s1Marks = {50, 30, 90};
int[] s2Marks = {60, 40, 80};
int[] s3Marks = {70, 50, 70};
```

This works, but imagine 50 students. You'd have 50 separate arrays. And how would you loop over all of them? This is exactly the problem arrays solved for single values.

The solution: A 2D array — an **array of arrays**.

---

11. 2D ARRAYS — CONCEPTUAL REPRESENTATION

A 2D array is visualized as a **matrix** (grid) with **rows** and **columns**:

```
        Column 0   Column 1   Column 2
         (Hindi)  (English)    (CS)
Row 0 (S1): [ 50 ]   [ 30 ]   [ 90 ]
Row 1 (S2): [ 60 ]   [ 40 ]   [ 80 ]
Row 2 (S3): [ 70 ]   [ 50 ]   [ 70 ]
```

- **Rows** are the horizontal lines (each row = one student's data)
- **Columns** are the vertical lines (each column = one subject)
- Each cell is identified by `[row][column]`

Indexing the cells:

```
marks[0][0] = 50   (S1's Hindi)
marks[0][1] = 30   (S1's English)
marks[0][2] = 90   (S1's CS)
marks[1][0] = 60   (S2's Hindi)
marks[1][1] = 40   (S2's English)
marks[1][2] = 80   (S2's CS)
marks[2][0] = 70   (S3's Hindi)
marks[2][1] = 50   (S3's English)
marks[2][2] = 70   (S3's CS)
```

The rule: First bracket = **row**, second bracket = **column**

---

12. DECLARING & DEFINING A 2D ARRAY

int[][] marks = new int[3][3];

Breaking it down:

Part	Meaning
`int`	Type of each final element
`[][]`	Two brackets = 2 dimensions
`marks`	Name of the array

`new`	Allocate memory in Heap
`int[3]`	Outer array has 3 elements
`[3]`	Each inner array has 3 elements

Inserting values manually:

```
marks[0][0] = 50;
marks[0][1] = 30;
marks[0][2] = 90;
marks[1][0] = 60;
marks[1][1] = 40;
marks[1][2] = 80;
marks[2][0] = 70;
marks[2][1] = 50;
marks[2][2] = 70;
```

---

13. CONCEPTUAL VS LOGICAL REPRESENTATION — THE CRUCIAL DISTINCTION

This is one of the most important concepts to understand, and most tutorials skip it entirely.

Conceptual Representation (How WE think about it)

We imagine a nice 2D grid like this:

```
[50][30][90]
[60][40][80]
[70][50][70]
```

This is a convenient mental model. It helps humans reason about the data.

Logical/Physical Representation (How the COMPUTER actually stores it)

The computer has no concept of "rows" and "columns." It only knows about **memory addresses** and **contiguous blocks**.

When you write `int[][] marks = new int[3][3]`, here's what actually happens in memory:

Step 1: Java creates an outer array of size 3:

Outer array "marks":
[ref0] [ref1] [ref2]
  ↓ ↓ ↓

Each element of this outer array doesn't hold an `int` — it holds a **reference (memory address)** to another array!

Step 2: Java creates 3 inner arrays, each of size 3:

Inner array 0 (marks[0]): [50][30][90]
Inner array 1 (marks[1]): [60][40][80]
Inner array 2 (marks[2]): [70][50][70]

The complete picture:

```
**The complete picture:**
```

```
STACK:                HEAP:
marks ──────────►  Outer Array:
                   [addr_A][addr_B][addr_C]
                       |       |       |
                       ▼       ▼       ▼
                   [50,30,90] [60,40,80] [70,50,70]
                   (marks[0]) (marks[1]) (marks[2])
```

So `marks[1][2]` works like this:

1. Go to `marks` → find address of outer array
2. Go to `marks[1]` → find address of the second inner array (`addr_B`)
3. Go to `marks[1][2]` → go to index 2 of that inner array → get `80`

This is why the lecturer says "marks[0] is itself an array" — because it literally is. It's not just a row in a table. It's a full array object in heap memory.

---

14. NESTED LOOPS FOR 2D ARRAYS

To traverse a 2D array, you need a **nested loop** — a loop inside another loop.

```
for (int row = 0; row < marks.length; row++) {
    for (int col = 0; col < marks[row].length; col++) {
        System.out.print(marks[row][col] + " ");
    }
    System.out.println();  // New line after each row
}
```

Why `marks.length` for outer loop?

- `marks` is the outer array
- `marks.length` = 3 (number of rows)
- So outer loop runs for rows: 0, 1, 2

Why `marks[row].length` for inner loop?

- `marks[row]` is the inner array for that specific row
- `marks[0].length` = 3, `marks[1].length` = 3, `marks[2].length` = 3
- The inner loop runs for each column in that row

Why NOT just `marks.length` for both loops?

- For a regular square matrix it would work (both are 3)
- But for **jagged arrays** (where each row has different number of columns), you MUST use `marks[row].length` for the inner loop
- Using `marks[row].length` is always correct; using `marks.length` for inner loop is only sometimes correct

Detailed trace of the nested loop:

```
Outer loop i=0:
  Inner loop j=0: print marks[0][0] = 50
  Inner loop j=1: print marks[0][1] = 30
  Inner loop j=2: print marks[0][2] = 90
  Inner loop done. Print newline.
```

```
  Output so far: "50 30 90"

Outer loop i=1:
  Inner loop j=0: print marks[1][0] = 60
  Inner loop j=1: print marks[1][1] = 40
  Inner loop j=2: print marks[1][2] = 80
  Inner loop done. Print newline.
  Output so far: "60 40 80"

Outer loop i=2:
  Inner loop j=0: print marks[2][0] = 70
  Inner loop j=1: print marks[2][1] = 50
  Inner loop j=2: print marks[2][2] = 70
  Inner loop done. Print newline.
  Output so far: "70 50 70"
```

---

15. JAGGED ARRAYS (DIFFERENT LENGTH ROWS)

A **jagged array** (also called a ragged array) is a 2D array where each row can have a **different number of columns**.

Real-world scenario: Students enrolled in different numbers of subjects:

- Student 1: enrolled in 1 subject
- Student 2: enrolled in 2 subjects
- Student 3: enrolled in 3 subjects

How to create a jagged array:

```
// Step 1: Create the outer array (specify number of rows only)
int[][] marks = new int[3][];
//                      ^^^ No column size here! Left blank intentionally.
```

At this point:

marks → [null][null][null]

Each row is `null` — it exists as a slot but hasn't been given an inner array yet.

```
// Step 2: Create each inner array with its own size
marks[0] = new int[1];  // Row 0 has 1 column
```

```
marks[1] = new int[2];  // Row 1 has 2 columns
marks[2] = new int[3];  // Row 2 has 3 columns
```

Now:

```
marks[0] → [0]          (1 element)
marks[1] → [0][0]       (2 elements)
marks[2] → [0][0][0]    (3 elements)

// Step 3: Assign values
marks[0][0] = 23;

marks[1][0] = 24;
marks[1][1] = 90;

marks[2][0] = 12;
marks[2][1] = 78;
marks[2][2] = 45;

// Step 4: Print — MUST use marks[row].length for inner loop!
for (int row = 0; row < marks.length; row++) {
    for (int col = 0; col < marks[row].length; col++) {  // ← Different each row!
        System.out.print(marks[row][col] + " ");
    }
    System.out.println();
}
```

Output:

```
23
24 90
12 78 45
```

If you mistakenly used `marks.length` for both:

```
for (int col = 0; col < marks.length; col++) {  // marks.length = 3 always
    // When row=0, col goes 0,1,2 — but marks[0] only has index 0!
    // marks[0][1] → ArrayIndexOutOfBoundsException!
}
```

This is exactly why the logical representation (understanding that each row IS an array with its own `.length`) matters.

---

16. SHORTHAND ARRAY INITIALIZATION

You can declare, define, and initialize all in one line:

1D Array shorthand:

```
// Long form:
int[] arr = new int[3];
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;

// Shorthand:
int[] arr = {10, 20, 30};
```

Both are identical in effect. The shorthand:

- Infers size from number of values (3 values → size 3)
- Assigns values immediately
- No `new int[3]` needed

2D Array shorthand:

```
// Long form:
int[][] marks = new int[3][3];
marks[0][0] = 50; marks[0][1] = 30; marks[0][2] = 90;
marks[1][0] = 60; marks[1][1] = 40; marks[1][2] = 80;
marks[2][0] = 70; marks[2][1] = 50; marks[2][2] = 70;

// Shorthand:
int[][] marks = {
    {50, 30, 90},  // Row 0 — its own array
    {60, 40, 80},  // Row 1 — its own array
    {70, 50, 70}   // Row 2 — its own array
};
```

The inner `{}` braces each create one inner array. This shorthand can even create jagged arrays automatically:

```
int[][] jagged = {
    {10},           // Row 0: 1 element
    {20, 30},       // Row 1: 2 elements
    {40, 50, 60}    // Row 2: 3 elements
};
```

---

17. 3D ARRAYS & HIGHER DIMENSIONS

A 3D array is an **array of 2D arrays** — or more precisely, an array whose elements are arrays, whose elements are arrays of ints.

Declaration:

```
int[][][] cube = new int[3][3][3];
```

Conceptual visualization (physical cube):

Think of a Rubik's cube:

- The outermost dimension = which "layer" (front-to-back)
- Middle dimension = which row within that layer
- Inner dimension = which column

```
Layer 0:        Layer 1:        Layer 2:
[a][b][c]       [j][k][l]       [s][t][u]
[d][e][f]       [m][n][o]       [v][w][x]
[g][h][i]       [p][q][r]       [y][z][!]
```

Logical representation (how computer stores it):

```
cube → outer array [ref0][ref1][ref2]
       ↓
ref0 → 2D array (Layer 0): [[a,b,c],[d,e,f],[g,h,i]]
ref1 → 2D array (Layer 1): [[j,k,l],[m,n,o],[p,q,r]]
ref2 → 2D array (Layer 2): [[s,t,u],[v,w,x],[y,z,!]]
```

Each inner 2D array is itself an array of arrays.

Traversal (triple nested loop):

```
for (int layer = 0; layer < cube.length; layer++) {
```

```
    for (int row = 0; row < cube[layer].length; row++) {
        for (int col = 0; col < cube[layer][row].length; col++) {
            System.out.print(cube[layer][row][col] + " ");
        }
        System.out.println();
    }
    System.out.println("--- Layer " + layer + " end ---");
}
```

When are 3D arrays used?

- **3D graphics:** representing voxels in a 3D space
- **Simulations:** weather data (latitude × longitude × altitude)
- **Video frames:** (frame number × height × width) for video processing
- **Competitive programming:** some DP problems require 3D state

In practice, 4D and above are extremely rare because they become mentally unmanageable. But syntactically, Java allows any number of dimensions: `int[][][][][]` etc.

---

18. COMPLETE CODE EXAMPLES

```
public class ArraysDemo {
    public static void main(String[] args) {

        // ============ 1D ARRAY ============

        // Declaration + Definition + Initialization
        int[] rollNumbers = new int[3];
        rollNumbers[0] = 1001;
        rollNumbers[1] = 1002;
        rollNumbers[2] = 1003;

        // Print using loop
        System.out.println("--- 1D Array ---");
        for (int i = 0; i < rollNumbers.length; i++) {
            System.out.println(rollNumbers[i]);
        }

        // Shorthand
        int[] scores = {85, 92, 78, 95, 88};
        System.out.println("Array length: " + scores.length);  // 5
```

```
// ============ 2D ARRAY ============

int[][] marks = {
    {50, 30, 90},
    {60, 40, 80},
    {70, 50, 70}
};

System.out.println("\n--- 2D Array ---");
for (int row = 0; row < marks.length; row++) {
    for (int col = 0; col < marks[row].length; col++) {
        System.out.print(marks[row][col] + "\t");
    }
    System.out.println();
}


// ============ JAGGED ARRAY ============

int[][] jagged = new int[3][];
jagged[0] = new int[]{23};
jagged[1] = new int[]{24, 90};
jagged[2] = new int[]{12, 78, 45};

System.out.println("\n--- Jagged Array ---");
for (int row = 0; row < jagged.length; row++) {
    for (int col = 0; col < jagged[row].length; col++) {
        System.out.print(jagged[row][col] + " ");
    }
    System.out.println();
}


// ============ 3D ARRAY ============

int[][][] cube = new int[2][2][2];
int val = 1;
for (int l = 0; l < cube.length; l++) {
    for (int r = 0; r < cube[l].length; r++) {
        for (int c = 0; c < cube[l][r].length; c++) {
            cube[l][r][c] = val++;
        }
```

```
        }
    }

    System.out.println("\n--- 3D Array ---");
    for (int l = 0; l < cube.length; l++) {
        System.out.println("Layer " + l + ":");
        for (int r = 0; r < cube[l].length; r++) {
            for (int c = 0; c < cube[l][r].length; c++) {
                System.out.print(cube[l][r][c] + " ");
            }
            System.out.println();
        }
    }
  }
}
```

---

19. INTRODUCTION TO STRINGS — COMPLETE DEPTH

What is a String?

A **String** is a sequence of characters. It is **not a primitive type** in Java — it is an **object** (an instance of the `java.lang.String` class).

String name = "Aditya";

- `String` — the class name (note capital S — it's a class, not a primitive)
- `name` — variable name (reference)
- `"Aditya"` — string literal (always in double quotes)

Characters that can be in a String:

- Letters: `A-Z`, `a-z`
- Digits: `0-9`
- Symbols: `!@#$%^&*()` etc.
- Spaces: `"  "` (a space is a valid character)
- Even an empty string: `""` (zero characters)

String vs char

Feature	`char`	`String`
What it holds	One character	Zero or more characters
Quotes used	Single: `'A'`	Double: `"Aditya"`
Data type category	Primitive	Non-primitive (Object)
Size	16 bits (2 bytes)	Depends on content

```
char c = 'A';          // single character, single quotes
String s = "Aditya";   // multiple characters, double quotes
String empty = "";     // zero characters — still valid!
String space = " ";    // just a space — also valid!
```

How Strings are stored in memory

Strings in Java are stored in a special area of the Heap called the **String Pool** (also called String Constant Pool).

```
String s1 = "Hello";
String s2 = "Hello";
```

Both `s1` and `s2` actually point to the **same** object in the String Pool! Java doesn't create two copies of "Hello" — it reuses the existing one. This is called **String interning**.

```
String s3 = new String("Hello");  // Forces creating a NEW object, not from pool
```

This is why comparing Strings with `==` can give unexpected results (it compares references, not content). Always use `.equals()` to compare String content.

String Concatenation

Concatenation means joining two strings end-to-end.

```
String firstName = "Aditya";
String lastName = "Tandon";

// Method 1: + operator
String fullName = firstName + " " + lastName;
System.out.println(fullName);  // "Aditya Tandon"
```

```
// Method 2: Concatenating in print directly
System.out.println(firstName + " " + lastName);  // "Aditya Tandon"
```

Important: + behaves differently for Strings vs numbers!

```
int a = 5, b = 3;
System.out.println(a + b);        // 8 (numeric addition)

String x = "5", y = "3";
System.out.println(x + y);        // "53" (string concatenation!)

System.out.println("Value: " + 5 + 3);   // "Value: 53" — tricky!
System.out.println("Value: " + (5 + 3)); // "Value: 8" — correct!
```

The rule: When + involves a String, all + operations from that point are treated as concatenation, left to right. Use parentheses to force numeric addition first.

Strings are Immutable

Once a String is created, **it cannot be changed**. This is called **immutability**.

```
String s = "Hello";
s = s + " World";  // This doesn't modify "Hello" — it creates a NEW string "Hello World"
                   // and makes s point to the new one
                   // "Hello" still exists in memory (until garbage collected)
```

This is why for heavy string manipulation (building strings in loops), you should use `StringBuilder` instead — but that's an advanced topic.

Common String Operations (Preview)

```
String s = "Hello World";

s.length()          // 11 — number of characters
s.charAt(0)         // 'H' — character at index 0
s.toUpperCase()     // "HELLO WORLD"
s.toLowerCase()     // "hello world"
s.contains("World")  // true
s.substring(6)      // "World" — from index 6 to end
s.substring(0, 5)   // "Hello" — from index 0 to 4
s.equals("Hello World")     // true (compare content)
```

```
s.equalsIgnoreCase("hello world")  // true (case-insensitive)
s.replace("World", "Java")   // "Hello Java"
s.trim()            // removes leading/trailing spaces
s.split(" ")        // returns String[] {"Hello", "World"}
```

These will be explored in depth after OOP concepts are covered.

Why String is Non-Primitive (Object)?

Primitive types like `int`, `char` etc. just store raw values. But `String`:

- Has **methods** you can call (`.length()`, `.charAt()`, etc.)
- Has complex internal behavior (immutability, String pool, etc.)
- Is defined as a **class** in Java's standard library
- Variables hold a **reference** to the String object, not the string itself

This is why we write `String` with a capital `S` — in Java, class names by convention start with uppercase.

QUICK SUMMARY TABLE

Concept	Key Point
Why arrays	Store multiple values of same type under one name
Contiguous memory	Elements stored back-to-back, enabling O(1) access
Zero-based indexing	First element at index 0, last at index n-1
`new` keyword	Allocates memory on Heap
`.length` property	Gives array size; no parentheses; adapts to any size
`ArrayIndexOutOfBoundsException`	Accessing invalid index at runtime
2D array	Array of arrays; first `[]` = rows, second `[]` = columns
Conceptual view	Grid/matrix (how humans think)
Logical view	Outer array holding references to inner arrays (how computer stores)

`marks[row].length`	Inner loop must use this, not `marks.length`, for correctness
Jagged array	2D array where each row has different number of columns
3D array	Array of 2D arrays; visualized as a cube
String	Non-primitive, object, immutable sequence of characters, double quotes
String concatenation	`+` operator joins strings end-to-end

This covers every single concept from the lecture in complete depth. Every "why" has been answered, every internal mechanism has been explained, and every common pitfall has been highlighted.

Everything NOT Covered — Complete Deep Dive

Let me identify and explain every concept that was either glossed over, mentioned briefly, or completely missing from the previous explanation.

---

1. THE HEAP vs STACK — COMPLETE INTERNAL STORY

The previous explanation mentioned these briefly. Let me go fully deep.

What is RAM (Memory)?

Your computer's RAM is just a giant sequence of bytes, each with a unique address. When a Java program runs, the JVM carves this RAM into several distinct regions:

```
┌──────────────────────────────────────┐
│            METHOD AREA               │ ← Stores class definitions, static
variables
├──────────────────────────────────────┤
│               HEAP                   │ ← All objects & arrays live here
├──────────────────────────────────────┤
│               STACK                  │ ← Method calls, local variables live here
├──────────────────────────────────────┤
│           PC REGISTERS               │ ← Tracks which instruction is executing
├──────────────────────────────────────┤
│        NATIVE METHOD STACK           │ ← For non-Java code
└──────────────────────────────────────┘
```

The Stack — Deep Explanation

The **Stack** is organized as a collection of **stack frames**. Every time a method is called, a new frame is pushed onto the stack. When the method finishes, that frame is popped off (destroyed).

```
public static void main(String[] args) {
    int x = 5;                  // x lives in main's stack frame
    int[] arr = new int[3];       // arr (the reference) lives in main's stack frame
}
```

What a stack frame contains:

- Local variables
- Method parameters
- Return address (where to go after method finishes)
- Reference to the current object (`this`) if applicable

Stack is LIFO (Last In, First Out). The most recently called method is always on top.

Stack memory is extremely fast because allocation/deallocation is just moving a pointer up or down. But it's also **limited in size** — typically 512KB to 1MB. This is why infinite recursion causes a `StackOverflowError`.

The Heap — Deep Explanation

The **Heap** is a large, unstructured pool of memory. All objects and arrays live here because:

1. Their size might not be known at compile time
2. They might need to outlive the method that created them

3. They can be large (stack is too small)

Heap memory is managed by the Garbage Collector (GC). When no variable on the Stack references a Heap object anymore, the GC reclaims that memory automatically.

```
public static void main(String[] args) {
   int[] arr = new int[1000000];  // 4MB on Heap — fine!
   // arr goes out of scope here → GC will eventually free those 4MB
}
```

Stack vs Heap side by side:

Feature	Stack	Heap
What lives there	Primitive variables, references	Objects, arrays
Size	Small (KBs)	Large (GBs)
Management	Automatic (method call/return)	Garbage Collector
Speed	Very fast	Slower
Access	LIFO (structured)	Random (unstructured)
Thread safety	Each thread has its own stack	Shared across all threads
Error when full	StackOverflowError	OutOfMemoryError

---

2. WHAT EXACTLY IS A REFERENCE? (Pointer concept)

The word "reference" was used but never truly explained.

In C/C++, they call it a Pointer

```
int x = 5;
int* ptr = &x;  // ptr stores the MEMORY ADDRESS of x
```

In Java, it's called a Reference

Java hides the raw memory address from you, but the concept is identical. A reference is a variable that **stores the location (address) of an object** in heap memory.

```
int[] arr = new int[3];
```

What `arr` actually contains is NOT the array data. It contains something like `0x7f4a8b3c` — a hexadecimal memory address pointing to where the array is in the Heap.

Proof — what happens when you assign one array to another:

```
int[] arr1 = {1, 2, 3};
int[] arr2 = arr1;  // arr2 gets the SAME address as arr1

arr2[0] = 999;  // Modifying through arr2...

System.out.println(arr1[0]);  // prints 999 !!
```

This happens because both `arr1` and `arr2` point to the **same array in memory**. You haven't created a copy — you've created a second reference to the same object.

```
Stack:                          Heap:
arr1 ──────┐
           ├──────────► [999][2][3]
arr2 ──────┘
```

To actually copy an array:

```
int[] arr1 = {1, 2, 3};
int[] arr2 = new int[arr1.length];

for (int i = 0; i < arr1.length; i++) {
    arr2[i] = arr1[i];
}
// OR use: arr2 = Arrays.copyOf(arr1, arr1.length);
```

Now they are independent:

```
Stack:                  Heap:
arr1 ──────────────► [1][2][3]
arr2 ──────────────► [1][2][3]  ← separate copy
```

---

3. NULL — WHAT IT ACTUALLY IS

When you declare a reference variable without assigning it:

```
int[] arr;  // Declaration only — what is arr?
```

`arr` is `null`. In Java, `null` means **"this reference points to nothing"** — it holds no valid memory address.

```
int[] arr = null;  // Explicitly null
System.out.println(arr.length);  // NullPointerException!
```

NullPointerException is one of the most common Java errors. It happens when you try to use a reference that points to nothing.

When does null appear with arrays?

```
int[][] jagged = new int[3][];
// jagged[0], jagged[1], jagged[2] are all null here
// because we haven't assigned inner arrays yet

System.out.println(jagged[0].length);  // NullPointerException!
// jagged[0] is null — it points to nothing!

// Fix:
jagged[0] = new int[2];
System.out.println(jagged[0].length);  // 2 — now it works
```

---

4. THE `new` KEYWORD — COMPLETE STORY

The lecture said "just use `new` as a black box for now." Let me open that black box.

`new` does four things in one shot:

Step 1: Allocates memory on the Heap Finds a contiguous block large enough for the object/array being created.

Step 2: Initializes to default values Every slot gets zeroed out (0 for int, false for boolean, null for references, etc.)

Step 3: Calls the constructor (for objects) For arrays, there's no constructor — allocation and initialization are enough.

Step 4: Returns a reference Returns the memory address of the newly created object, which gets stored in your variable.

```
int[] arr = new int[3];
//          ^^^^^^^^^^^
//     1. Find 12 bytes on Heap
//     2. Fill with 0,0,0
//     3. N/A for arrays
//     4. Return address → store in arr
```

---

5. WHY `int` IS 32 BITS — THE BINARY EXPLANATION

The lecture kept saying "32 bits" without explaining why.

Bits and Bytes

- 1 **bit** = smallest unit of storage = 0 or 1
- 8 **bits** = 1 **byte**
- 32 bits = 4 bytes

Why `int` is 32 bits

A 32-bit space can hold 2^{32} different values = 4,294,967,296 values.

Since `int` can be negative too, it's split:

- Half for negatives: −2,147,483,648 to −1
- Half for positives: 0 to 2,147,483,647

That's why Java's `int` range is exactly **−2,147,483,648 to 2,147,483,647**.

All primitive type sizes:

Type	Bits	Bytes	Range/Info
`byte`	8	1	−128 to 127
`short`	16	2	−32,768 to 32,767
`int`	32	4	−2.1B to 2.1B

`long`	64	8	Very large numbers
`float`	32	4	Decimal, ~7 digits precision
`double`	64	8	Decimal, ~15 digits precision
`char`	16	2	0 to 65,535 (Unicode)
`boolean`	1*	1*	true or false

*boolean is conceptually 1 bit but JVM typically uses 1 byte for alignment

---

6. ARRAY SIZE IS FIXED — WHY AND WHAT HAPPENS INTERNALLY

The lecture stated arrays are fixed-size but never explained WHY they can't grow.

The Contiguity Problem

When you create `new int[3]`, memory looks like:

Address: 1000 1004 1008 1012 1016 1020
Content: [ARR0][ARR1][ARR2][USED][USED][USED]

Addresses 1012 onwards are already occupied by other data. You CANNOT just add a 4th element because:

1. You'd need address 1012 — which is taken
2. If you move the array elsewhere to find more space, all existing references would be broken (they'd still point to 1000)

This is why arrays are fixed-size. The memory is "locked in" at creation.

What Java provides instead

For dynamic (growable) collections:

- `ArrayList` — dynamically growing array under the hood
- `LinkedList` — linked nodes, no contiguity required

- `HashMap`, `HashSet` etc. for other structures

---

7. MEMORY ADDRESS ARITHMETIC — THE EXACT FORMULA

When the CPU accesses `arr[i]`, it doesn't search through memory. It calculates:

Address of arr[i] = BaseAddress + (i × sizeof(elementType))

For `int[] arr` starting at address 1000:

- `arr[0]` → 1000 + (0 × 4) = **1000**
- `arr[1]` → 1000 + (1 × 4) = **1004**
- `arr[7]` → 1000 + (7 × 4) = **1028**
- `arr[999]` → 1000 + (999 × 4) = **4996**

This is O(1) — constant time regardless of array size or index value. This is arrays' greatest strength.

---

8. THE FOR-EACH LOOP (Enhanced For Loop) — COMPLETELY MISSING

The previous explanation barely mentioned this. It's extremely important.

```
int[] arr = {10, 20, 30, 40, 50};

// Traditional for loop:
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}

// Enhanced for loop (for-each):
for (int element : arr) {
    System.out.println(element);
}
```

Reading it: "For each `int` named `element` in `arr`, do..."

When to use for-each vs traditional for:

Use for-each when:

- You only need to READ values (not write/modify them)
- You don't need the index number
- You want cleaner, more readable code

Use traditional for when:

- You need the index `i` for something
- You need to MODIFY array values (`arr[i] = newValue`)
- You need to iterate in reverse
- You need to skip elements

```
// LIMITATION of for-each — this does NOT modify the array:
for (int element : arr) {
    element = element * 2;  // Only modifies LOCAL copy, not arr!
}
// arr is unchanged!

// Traditional for DOES modify the array:
for (int i = 0; i < arr.length; i++) {
    arr[i] = arr[i] * 2;  // Modifies actual array element
}
```

For-each with 2D arrays:

```
int[][] matrix = {{1,2,3},{4,5,6},{7,8,9}};

for (int[] row : matrix) {        // each row is an int[]
    for (int element : row) {     // each element in that row
        System.out.print(element + " ");
    }
    System.out.println();
}
```

---

9. `Arrays` CLASS — COMPLETELY MISSING

Java has a built-in utility class `java.util.Arrays` with extremely useful methods. This was never mentioned.

```
import java.util.Arrays;

int[] arr = {5, 3, 1, 4, 2};

// Sort the array
Arrays.sort(arr);
System.out.println(Arrays.toString(arr));  // [1, 2, 3, 4, 5]

// Search in sorted array (binary search)
int index = Arrays.binarySearch(arr, 3);  // returns 2

// Fill entire array with one value
Arrays.fill(arr, 7);  // [7, 7, 7, 7, 7]

// Copy an array
int[] copy = Arrays.copyOf(arr, arr.length);
int[] partial = Arrays.copyOfRange(arr, 1, 4); // copies index 1,2,3

// Compare two arrays
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};
System.out.println(a == b);           // false! (compares references)
System.out.println(Arrays.equals(a, b)); // true! (compares contents)

// Print array contents easily
System.out.println(arr);              // prints something like [I@7852e922 (useless!)
System.out.println(Arrays.toString(arr)); // prints [1, 2, 3, 4, 5] (useful!)

// For 2D arrays:
System.out.println(Arrays.deepToString(matrix)); // [[1,2],[3,4]]
```

---

10. PRINTING AN ARRAY DIRECTLY — WHY IT GIVES GARBAGE

This is a very common beginner confusion.

```
int[] arr = {1, 2, 3};
System.out.println(arr);  // prints something like: [I@6d06d69c
```

Why?

Because `System.out.println` calls `.toString()` on the object. For arrays, the default `.toString()` returns:

- `[` — opening bracket indicating array
- `I` — the type code (`I` = int, `B` = byte, `J` = long, `Ljava.lang.String;` = String array, etc.)
- `@` — separator
- `6d06d69c` — hexadecimal memory address (hashcode)

To print array contents correctly:

```
// Option 1: Arrays.toString()
System.out.println(Arrays.toString(arr));        // [1, 2, 3]

// Option 2: Manual loop
for (int x : arr) System.out.print(x + " ");    // 1 2 3

// Option 3: For 2D arrays
System.out.println(Arrays.deepToString(matrix)); // [[1,2],[3,4]]
```

---

11. PASSING ARRAYS TO METHODS — COMPLETELY MISSING

This is a crucial concept not covered at all.

Passing a primitive vs passing an array:

```
// Primitive — pass by VALUE
public static void changePrimitive(int x) {
    x = 999;  // Only changes LOCAL copy
}

int num = 5;
changePrimitive(num);
System.out.println(num);  // Still 5 — original unchanged!

// Array — pass by REFERENCE
public static void changeArray(int[] arr) {
```

```
    arr[0] = 999;  // Changes the ACTUAL array in Heap!
}

int[] myArr = {1, 2, 3};
changeArray(myArr);
System.out.println(myArr[0]);  // 999 — original WAS changed!
```

Why?

When you pass an array to a method, you pass a **copy of the reference** (the address). Both the original variable and the parameter now point to the **same array in Heap**. So any modification through the parameter affects the original.

```
Before calling changeArray(myArr):
Stack:
myArr ──────────────────────────────────────► [1][2][3]  (Heap)

After entering changeArray:
Stack:
myArr ──────────────────────────────────────► [1][2][3]  (Heap)
arr (parameter) ────────────────────────────^ (same location!)

After arr[0] = 999:
Stack:
myArr ──────────────────────────────────────► [999][2][3]  (Heap, modified!)
arr (parameter) ────────────────────────────^
```

Returning arrays from methods:

```
public static int[] createArray(int size) {
    int[] result = new int[size];
    for (int i = 0; i < size; i++) {
        result[i] = i * 10;
    }
    return result;  // Returns the reference
}

int[] myArr = createArray(5);
// myArr now points to [0, 10, 20, 30, 40]
```

---

12. TIME COMPLEXITY OF ARRAY OPERATIONS — NOT MENTIONED AT ALL

Understanding performance is crucial.

Operation	Time Complexity	Why
Access `arr[i]`	**O(1)**	Direct address calculation
Search (unsorted)	**O(n)**	Must check each element
Search (sorted, binary search)	**O(log n)**	Halves search space each step
Insert at end (if space)	**O(1)**	Just set `arr[size] = value`
Insert at middle	**O(n)**	Must shift all elements right
Delete from middle	**O(n)**	Must shift all elements left
Delete from end	**O(1)**	Just reduce size counter

Why inserting in middle is O(n):

```
// Insert value 99 at index 2 in {1, 2, 3, 4, 5}
// Result should be: {1, 2, 99, 3, 4, 5}

// You must shift EVERY element from index 2 onwards, one position right:
// {1, 2, 3, 4, 5, ?}  → Move 5 to index 5
// {1, 2, 3, 4, 4, 5}  → Move 4 to index 4 (overwrote old 5)
// {1, 2, 3, 3, 4, 5}  → Move 3 to index 3
// {1, 2, 3, 3, 4, 5}  → Now write 99 at index 2
// {1, 2, 99, 3, 4, 5}
```

This shifting is why arrays are bad for frequent insertions/deletions in the middle.

---

13. WHAT HAPPENS WHEN `new` RUNS OUT OF MEMORY

```
int[] arr = new int[Integer.MAX_VALUE];  // Trying to allocate ~8 GB!
```

If the JVM can't find enough contiguous space in Heap:

Exception in thread "main" java.lang.OutOfMemoryError: Java heap space

How to give JVM more heap space:

```
java -Xmx4g MyProgram   # Set max heap to 4 GB
java -Xms512m MyProgram # Set initial heap to 512 MB
```

---

14. COMPILE-TIME vs RUNTIME ERRORS — NOT EXPLAINED

The lecture mentioned exceptions but didn't explain this important distinction.

Compile-time errors (caught before running):

```
int[] arr = new int[3];
arr["hello"] = 5;  // ERROR: index must be int, not String
                   // Caught by compiler — program won't even run
```

Runtime errors / Exceptions (happen while running):

```
int[] arr = new int[3];
int i = 5;
arr[i] = 10;  // Compiler can't know i=5 at compile time
              // Looks fine syntactically
              // But at RUNTIME, i=5 is out of bounds → Exception
```

Why this distinction matters:

- **Compile-time errors** = typos, wrong types, syntax mistakes. Fix before running.
- **Runtime errors** = logical mistakes that only show up with specific inputs. Must test thoroughly.

---

15. THE EXCEPTION HIERARCHY — WHAT THE LECTURE SKIPPED

The lecture said "exceptions are Java's way of saying something went wrong" — but didn't explain the structure.

```
Throwable
├── Error (serious JVM problems — usually unrecoverable)
│   ├── StackOverflowError
│   ├── OutOfMemoryError
│   └── ...
└── Exception (recoverable problems)
    ├── RuntimeException (unchecked — don't need to declare)
    │   ├── ArrayIndexOutOfBoundsException
    │   ├── NullPointerException
    │   ├── ClassCastException
    │   ├── NumberFormatException
    │   └── ...
    └── Checked Exceptions (must handle or declare)
        ├── IOException
        ├── SQLException
        └── ...
```

`ArrayIndexOutOfBoundsException` is a `RuntimeException`, which means:

- You don't HAVE to handle it (though you should)
- It's unchecked — compiler won't force you to catch it
- It crashes the program if not caught

---

16. TRY-CATCH — HANDLING EXCEPTIONS (Briefly, since full topic is later)

The lecture mentioned exceptions but never showed how to handle them.

```
int[] arr = {1, 2, 3};

try {
    System.out.println(arr[5]);  // This will throw exception
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Index doesn't exist! " + e.getMessage());
} finally {
    System.out.println("This always runs, with or without exception");
}
```

Output:

Index doesn't exist! Index 5 out of bounds for length 3
This always runs, with or without exception

---

17. `String` IS IMMUTABLE — DEEP EXPLANATION

This was mentioned briefly but the implications weren't fully explored.

```
String s = "Hello";
s = s + " World";
```

What people think happens:

s: "Hello" → becomes → "Hello World"

What ACTUALLY happens:

1. "Hello" created in String Pool at address A
2. s points to A
3. "Hello World" created as NEW object at address B
4. s now points to B
5. "Hello" at address A still exists in memory (until GC cleans it)

Why immutability matters:

```
// This is EXTREMELY inefficient:
String result = "";
for (int i = 0; i < 10000; i++) {
    result = result + i;  // Creates 10000 new String objects!
}

// This is EFFICIENT:
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) {
    sb.append(i);  // Modifies same object each time
}
String result = sb.toString();  // One final String
```

`StringBuilder` is a mutable string — you modify it in place. This is called the **Builder pattern**.

---

18. STRING POOL — THE MEMORY OPTIMIZATION

```
String s1 = "Hello";   // Goes into String Pool
String s2 = "Hello";   // Reuses SAME object from Pool

System.out.println(s1 == s2);       // true! (same memory address)
System.out.println(s1.equals(s2));  // true! (same content)

String s3 = new String("Hello");    // Forces new Heap object, NOT from pool
System.out.println(s1 == s3);       // false! (different address)
System.out.println(s1.equals(s3));  // true! (same content)
```

Memory layout:

```
String Pool (special Heap area):
"Hello" ← both s1 and s2 point here

Regular Heap:
"Hello" ← s3 points here (a separate copy)
```

The golden rule: ALWAYS use `.equals()` to compare String content, NEVER `==`.

```
// == compares memory addresses (references)
// .equals() compares actual character content

String a = "test";
String b = "test";
String c = new String("test");

a == b       → true   (both from String Pool, same object)
a == c       → false  (c is on regular Heap, different object)
a.equals(c)  → true   (content is identical)
```

---

19. CHAR ARRAY vs STRING — RELATIONSHIP

A String internally is built on top of a `char` array. This is why String has similarities to arrays.

```
String s = "Hello";

// You can convert String to char array:
char[] chars = s.toCharArray();
// chars = ['H', 'e', 'l', 'l', 'o']

// Access individual characters:
System.out.println(s.charAt(0));  // 'H'
System.out.println(s.charAt(4));  // 'o'
// Note: charAt() is 0-indexed, just like arrays!

// You can convert char array back to String:
char[] letters = {'W', 'o', 'r', 'l', 'd'};
String word = new String(letters);  // "World"
```

Why this matters: Many string algorithms work by converting to `char[]`, manipulating, then converting back.

---

20. STRING COMPARISON — ALL METHODS

```
String s1 = "Hello";
String s2 = "hello";
String s3 = "Hello";

// == : Reference equality (same object?)
s1 == s3;              // true (both from String Pool)
s1 == new String("Hello"); // false

// .equals() : Content equality (same characters?)
s1.equals(s2);         // false ("Hello" ≠ "hello")
s1.equals(s3);         // true

// .equalsIgnoreCase() : Content equality, ignoring case
s1.equalsIgnoreCase(s2); // true ("Hello" == "hello" when ignoring case)

// .compareTo() : Lexicographic comparison (like dictionary order)
"Apple".compareTo("Banana"); // negative (A comes before B)
"Banana".compareTo("Apple"); // positive (B comes after A)
"Apple".compareTo("Apple");  // 0 (identical)
```

```
// Returns: negative if first < second, 0 if equal, positive if first > second
```

---

21. TYPE CASTING WITH ARRAYS — NOT MENTIONED

```
// You can cast numeric types, but you might lose data
double[] doubles = {1.9, 2.7, 3.5};
int[] ints = new int[3];

for (int i = 0; i < doubles.length; i++) {
    ints[i] = (int) doubles[i];  // Truncates decimal, doesn't round
    // 1.9 → 1, 2.7 → 2, 3.5 → 3
}
```

Object array type casting:

```
// Arrays follow the same type hierarchy as objects
Object[] objects = new String[3];  // Valid! String IS-A Object
objects[0] = "Hello";              // Valid
objects[1] = 42;                   // ArrayStoreException at runtime!
                                   // Can't put Integer into String[]
```

---

22. MULTI-DIMENSIONAL ARRAY — MEMORY LAYOUT DIAGRAM (EXACT)

Let me show the exact memory layout for `int[][] marks = new int[3][3]`:

```
STACK:
marks → [address of outer array: 0x100]

HEAP at 0x100 (outer array, 3 reference slots):
┌──────────────────┬──────────────────┬──────────────────┐
│ addr: 0x200      │ addr: 0x300      │ addr: 0x400      │
│ (marks[0])       │ (marks[1])       │ (marks[2])       │
└──────────────────┴──────────────────┴──────────────────┘

HEAP at 0x200 (marks[0], 3 int slots):
┌────┬────┬────┐
│ 50 │ 30 │ 90 │
└────┴────┴────┘

HEAP at 0x300 (marks[1], 3 int slots):
┌────┬────┬────┐
│ 60 │ 40 │ 80 │
└────┴────┴────┘

HEAP at 0x400 (marks[2], 3 int slots):
┌────┬────┬────┐
│ 70 │ 50 │ 70 │
└────┴────┴────┘
```

When you write `marks[1][2]`:

1. Go to marks → finds address 0x100
2. Go to 0x100, read slot [1] → finds address 0x300
3. Go to 0x300, read slot [2] → finds value 80

The outer array stores **references** (addresses), NOT actual int values. The inner arrays store **actual int values**. This is the fundamental structure.

---

23. WHY COLUMN SIZE IS OPTIONAL IN 2D ARRAY DECLARATION

```
int[][] arr = new int[3][];   // Valid — row size mandatory, column optional
int[][] arr = new int[][3];   // INVALID — compiler error!
```

Why must row size be specified?

The JVM needs to allocate the outer array immediately. It needs to know: "How many reference slots should I create?" Without the row count, it can't allocate even the outer array.

Why can column size be omitted?

The inner arrays can be created **later**, one by one. Their creation is deferred. So their sizes don't need to be known upfront.

```
int[][] arr = new int[3][];
// Outer array created: [null][null][null]
// Inner arrays: not created yet

arr[0] = new int[2];  // Now inner array 0 exists
arr[1] = new int[5];  // Now inner array 1 exists (different size!)
arr[2] = new int[1];  // Now inner array 2 exists
```

---

24. THE `System.out.println` IS ACTUALLY A STRING OPERATION

The lecture used `System.out.println` from day one, but never explained what it really is.

```
System.out.println("Hello");
```

Breaking it down:

- `System` — a class in `java.lang`
- `.out` — a static field of type `PrintStream`
- `.println()` — a method of `PrintStream`

When you pass things to `println`, it calls `.toString()` on them:

- `println(42)` → converts int 42 to String "42" → prints "42"
- `println(true)` → converts boolean to String "true" → prints "true"
- `println(arr)` → calls `arr.toString()` → gets garbage address like `[I@7f3`

String concatenation with `+`:

```
int x = 5;
System.out.println("Value is: " + x);
```

What Java actually does:

1. Sees `"Value is: "` — a String
2. Sees `+` — since left side is String, this is concatenation
3. Converts `x` (int 5) to String "5" automatically
4. Concatenates: "Value is: " + "5" = "Value is: 5"
5. Passes "Value is: 5" to println

This automatic int-to-String conversion is called **autoboxing + toString** under the hood.

---

25. VARIABLE SHADOWING IN LOOPS — SUBTLE BUG

```
int i = 100;  // Outer variable

for (int i = 0; i < 5; i++) {  // COMPILE ERROR in Java!
    // Java doesn't allow re-declaring i if it already exists in same scope
}
```

But this is fine:

```
for (int i = 0; i < 5; i++) {  // i declared in loop scope
    System.out.println(i);
}
// i here goes OUT of scope

for (int i = 0; i < 3; i++) {  // New i in new scope — perfectly fine
    System.out.println(i);
}
```

Each `for` loop creates its own scope. The `i` from loop 1 is completely gone by loop 2.

---

26. TWO WAYS TO WRITE ARRAY TYPE — SUBTLE DIFFERENCE

The lecture mentioned both but didn't explain which is preferred and why.

```
int[] arr;    // Java style — PREFERRED
int arr[];    // C/C++ style — allowed but not preferred
```

In Java, `int[]` is considered the **type** — "array of int." So:

```
// Java style — clear that BOTH a and b are int arrays:
int[] a, b;   // a is int[], b is int[]

// C style — ambiguous!
int a[], b;   // a is int[], but b is just int!
```

This is why the Java style is strongly preferred — it avoids ambiguity when declaring multiple variables.

---

27. ARRAY OF STRINGS — NOT SHOWN

```
String[] names = {"Alice", "Bob", "Charlie", "David"};

// Print all names
for (String name : names) {
    System.out.println(name);
}

// Find a name
String search = "Bob";
for (int i = 0; i < names.length; i++) {
    if (names[i].equals(search)) {  // .equals(), not ==
        System.out.println("Found " + search + " at index " + i);
    }
}
```

```
// Sort names alphabetically
Arrays.sort(names);
System.out.println(Arrays.toString(names));  // [Alice, Bob, Charlie, David]
```

Important: In a `String[]`, each element is a **reference** to a String object. So memory looks like:

```
HEAP (String array):
names → [ref0][ref1][ref2][ref3]
         ↓     ↓     ↓     ↓
       "Alice" "Bob" "Charlie" "David"  (String Pool)
```

---

28. ANONYMOUS ARRAYS — NOT MENTIONED

You can create an array without assigning it to a variable. Useful when passing arrays directly to methods.

```
// Normal way:
int[] arr = {1, 2, 3};
printArray(arr);

// Anonymous array — created inline, no variable:
printArray(new int[]{1, 2, 3});

// Another example:
System.out.println(Arrays.toString(new int[]{5, 10, 15}));
```

Why use anonymous arrays? When you need a temporary array for a single use — no need to create a named variable.

---

29. STRING METHODS — COMPLETE LIST NOT COVERED

```
String s = "  Hello, World!  ";

// Length
s.length()          // 17 (including spaces)
```

```
// Trimming whitespace
s.trim()            // "Hello, World!" (removes leading/trailing spaces)
s.strip()           // Same as trim but handles Unicode spaces too

// Case conversion
s.toUpperCase()      // "  HELLO, WORLD!  "
s.toLowerCase()      // "  hello, world!  "

// Checking content
s.isEmpty()          // false (has characters)
"".isEmpty()         // true
s.isBlank()          // false (has non-whitespace)
"   ".isBlank()      // true (only whitespace)
s.contains("World")  // true
s.startsWith("  H")  // true
s.endsWith("!  ")    // true

// Searching
s.indexOf("o")       // 4 (first occurrence)
s.lastIndexOf("o")   // 8 (last occurrence)
s.indexOf("xyz")     // -1 (not found → returns -1)

// Extracting parts
s.charAt(2)          // 'H' (0-indexed)
s.substring(2)       // "Hello, World!  " (from index 2 to end)
s.substring(2, 7)    // "Hello" (from index 2 to 6, end exclusive)

// Replacing
s.replace("World", "Java")  // "  Hello, Java!  "
s.replaceAll("\\s+", " ")   // Replace all whitespace sequences with single space

// Splitting
"a,b,c,d".split(",")         // ["a", "b", "c", "d"]
"Hello World".split(" ")     // ["Hello", "World"]
"one::two".split(":")        // ["one", "", "two"]

// Joining (static method)
String.join("-", "a", "b", "c")     // "a-b-c"
String.join(", ", names)            // joins array with comma

// Converting other types to String
String.valueOf(42)      // "42"
String.valueOf(3.14)    // "3.14"
String.valueOf(true)    // "true"
```

```
String.valueOf('A')     // "A"

// Converting String to other types
Integer.parseInt("42")      // 42 (int)
Double.parseDouble("3.14")  // 3.14 (double)
Boolean.parseBoolean("true") // true (boolean)

// Checking character types
Character.isDigit('5')      // true
Character.isLetter('A')     // true
Character.isWhitespace(' ') // true
Character.toUpperCase('a')  // 'A'
```

---

30. WHAT `static` MEANS IN `public static void main` — ALWAYS SKIPPED

Every Java program starts with:

```
public static void main(String[] args) {
```

The `args` array was never explained.

`String[] args` — Command Line Arguments

`args` is a **String array** that holds any arguments you pass when running the program from the command line.

```
java MyProgram Hello World 42
```

Inside the program:

```
public static void main(String[] args) {
    System.out.println(args.length);  // 3
    System.out.println(args[0]);      // "Hello"
    System.out.println(args[1]);      // "World"
    System.out.println(args[2]);      // "42"
}
```

Important: Everything in `args` is a String, even numbers. You must convert:

```
int number = Integer.parseInt(args[2]);  // Convert "42" to 42
```

---

31. HOW NESTED LOOPS ACTUALLY WORK — STEP BY STEP EXECUTION MODEL

Many people use nested loops without truly understanding the execution flow.

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        System.out.println("i=" + i + ", j=" + j);
    }
}
```

Complete execution trace:

```
i=0 starts
  j=0: print "i=0, j=0"    ← j goes 0,1,2 while i stays at 0
  j=1: print "i=0, j=1"
  j=2: print "i=0, j=2"
  j=3: j<3 is false, inner loop exits
i=0 ends, i++ → i=1
  j=0: print "i=1, j=0"    ← inner loop RESTARTS from j=0
  j=1: print "i=1, j=1"
  j=2: print "i=1, j=2"
  j=3: inner loop exits
i=1 ends, i++ → i=2
  j=0: print "i=2, j=0"
  j=1: print "i=2, j=1"
  j=2: print "i=2, j=2"
  j=3: inner loop exits
i=2 ends, i++ → i=3
i=3: 3<3 is false, outer loop exits
```

Total iterations: 3 × 3 = 9 for a 3×3 nested loop. For n×m, it's n×m total iterations.

For 3D arrays:

```
for (int l = 0; l < 3; l++) {
```

```
    for (int r = 0; r < 3; r++) {
        for (int c = 0; c < 3; c++) {
            // Runs 3 × 3 × 3 = 27 times
        }
    }
}
```

---

32. GARBAGE COLLECTION — WHAT HAPPENS TO OLD ARRAYS

When does memory get freed?

```
int[] arr1 = new int[1000];  // 4000 bytes allocated on Heap
arr1 = new int[5];           // arr1 now points to new array
                             // Old 1000-element array has NO references
                             // It becomes "garbage"
```

The **Garbage Collector (GC)** runs periodically and frees memory occupied by objects with no references. You don't do this manually in Java (unlike C/C++ where you call `free()`).

When does GC run?

- When JVM decides memory is running low
- You can suggest (not force): `System.gc()` — but JVM may ignore it

This matters for arrays because large arrays you're done with will eventually be freed automatically.

---

33. ARRAY COVARIANCE — ADVANCED BUT IMPORTANT

```
String[] strings = new String[3];
Object[] objects = strings;  // Valid! String[] IS-A Object[]
objects[0] = "Hello";        // Fine
objects[1] = 42;             // ArrayStoreException at RUNTIME!
                             // Can't store Integer in String[]
```

This is called **array covariance** — arrays are covariant in Java (unlike generics). It can cause runtime errors that the compiler can't catch at compile time. This is considered a design flaw in Java.

---

34. COMPLETE WORKED EXAMPLE — FINDING MAX/MIN IN ARRAY

Common operations the lecture didn't show:

```
int[] arr = {5, 3, 8, 1, 9, 2, 7, 4, 6};

// Find maximum
int max = arr[0];  // Start with first element as initial max
for (int i = 1; i < arr.length; i++) {
    if (arr[i] > max) {
        max = arr[i];
    }
}
System.out.println("Max: " + max);  // 9

// Find minimum
int min = arr[0];
for (int i = 1; i < arr.length; i++) {
    if (arr[i] < min) {
        min = arr[i];
    }
}
System.out.println("Min: " + min);  // 1

// Calculate sum and average
int sum = 0;
for (int x : arr) {
    sum += x;
}
double average = (double) sum / arr.length;
System.out.println("Sum: " + sum);         // 45
System.out.println("Average: " + average); // 5.0

// Reverse the array
for (int i = 0, j = arr.length - 1; i < j; i++, j--) {
    int temp = arr[i];
```

```
    arr[i] = arr[j];
    arr[j] = temp;
}
System.out.println(Arrays.toString(arr));  // [6, 4, 7, 2, 9, 1, 8, 3, 5]
```

FINAL SUMMARY OF EVERYTHING NEW

Topic	Why Important
Stack vs Heap in depth	Understand where variables/arrays actually live
References and what they really store	Understand why array assignment copies address, not data
null and NullPointerException	Most common Java error
`new` keyword — 4 steps	Understand what happens behind the scenes
Why int is 32 bits	Understand type sizes and memory calculation
Why arrays are fixed size	Contiguity prevents growth
For-each loop	Cleaner way to iterate when index not needed
`Arrays` utility class	sort, fill, copyOf, toString, equals
Printing array directly gives garbage	Common beginner confusion
Passing arrays to methods	Arrays are passed by reference — modifications persist
Time complexity	O(1) access, O(n) search, O(n) insert/delete middle
OutOfMemoryError	What happens when heap fills up
Compile-time vs Runtime errors	Different debugging approach for each
Exception hierarchy	Where ArrayIndexOutOfBoundsException sits
try-catch	How to handle exceptions gracefully
String immutability implications	Why `sb = sb + x` in loop is slow
String Pool	Why `==` fails for Strings
charAt vs array indexing	Strings behave like char arrays

All String methods	Complete toolkit
`String[] args` in main	Command line arguments
Nested loop execution model	Exactly how i and j relate
Garbage collection	Memory is freed automatically
Anonymous arrays	Creating arrays inline for one-time use
Array of Strings	Strings in arrays are references
Array covariance	Subtle runtime error source
Common algorithms	Max, min, sum, reverse